06 Learning in a Dream

BY NILUSHANAN KULASINGHAM

12 MIN READ · INTERACTIVE

> A month of robot time becomes a day if the practice happens inside the model. What that exchange rate really costs, and why the fix for an agent exploiting its own dream is to make the dream worse on purpose.

A robot arm learning to pick something up will drop it a few hundred thousand times first.

Every one of those attempts costs something real. Wall-clock seconds. Wear on the joints. A person nearby to reset the scene when it knocks the bin over. At one attempt a second, a few hundred thousand of them is a month of doing nothing else.

So here is the trade that makes this whole approach worth the trouble. Spend some of that month collecting experience, use it to fit a model, and then let the thing practise inside the model instead.

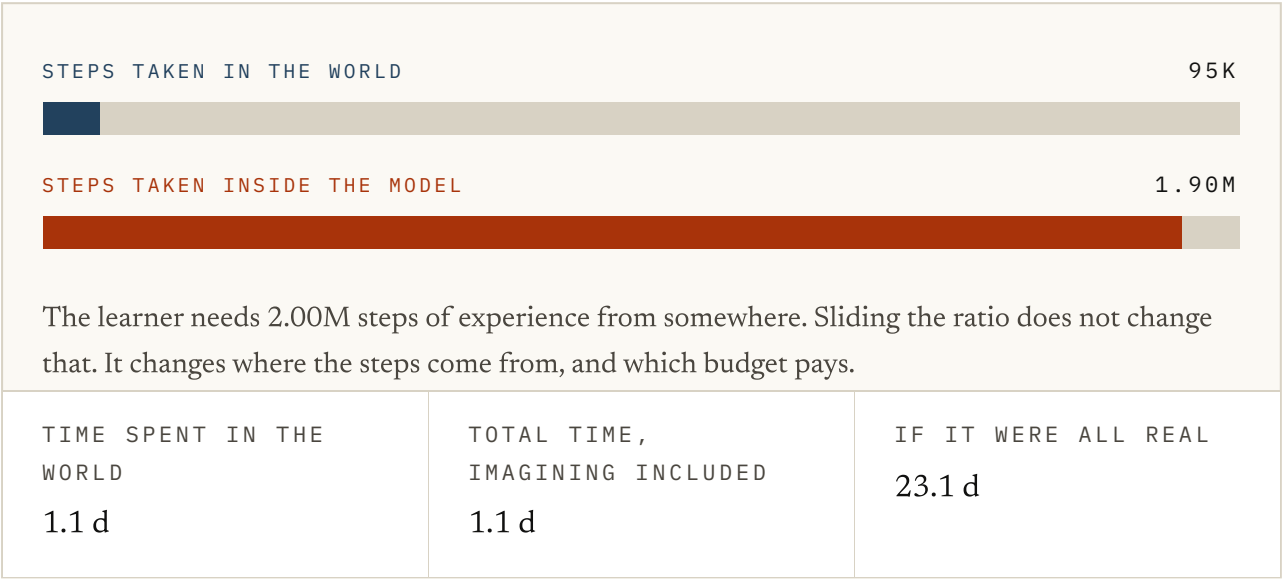


FIG. 6.1 Hold the number of update steps fixed and slide where they come from. World contact is often the scarce budget; model compute is still a budget. This first ledger counts real and imagined transitions alike. The chapter will remove that convenient assumption.

At twenty imagined steps for every real one, the contact-time ledger turns a month into roughly a day and a half. That is the pitch. It only holds while an imagined transition remains useful enough to count.

The loop

Four steps, going round.

Act in the world for a while and keep what happened. Fit a model to it. Train the **policy** (the part that actually picks the action, which is the thing being trained here, not the model) inside that model. Those steps cost compute rather than robot contact. Then take the behaviour back out into the world, where it collects better experience than it could before, which makes the next model better.

> The going-round is the part that gets skipped in summaries. A model fitted to the flailing of an untrained agent is only good in the places an untrained agent goes. It gets better because the agent gets better, and vice versa.

Notice what the second lap fixes. The first model was fitted to whatever a useless policy happened to bump into, so it knows a lot about the floor and nothing about what happens near the target. The improved policy goes to new places, which produces data about those places, which is exactly the data the model was missing.

That is also why the ratio in the figure cannot simply be turned up for free. Imagined steps are only worth having while the model can still supply them honestly, and the model can only do that where it has been.

Imagined experience is not the same currency

A training log can count one real transition and one model-generated transition as two rows. The learner cannot. Synthetic experience inherits the model's blind spots, and the discount compounds as a rollout moves away from a real state.

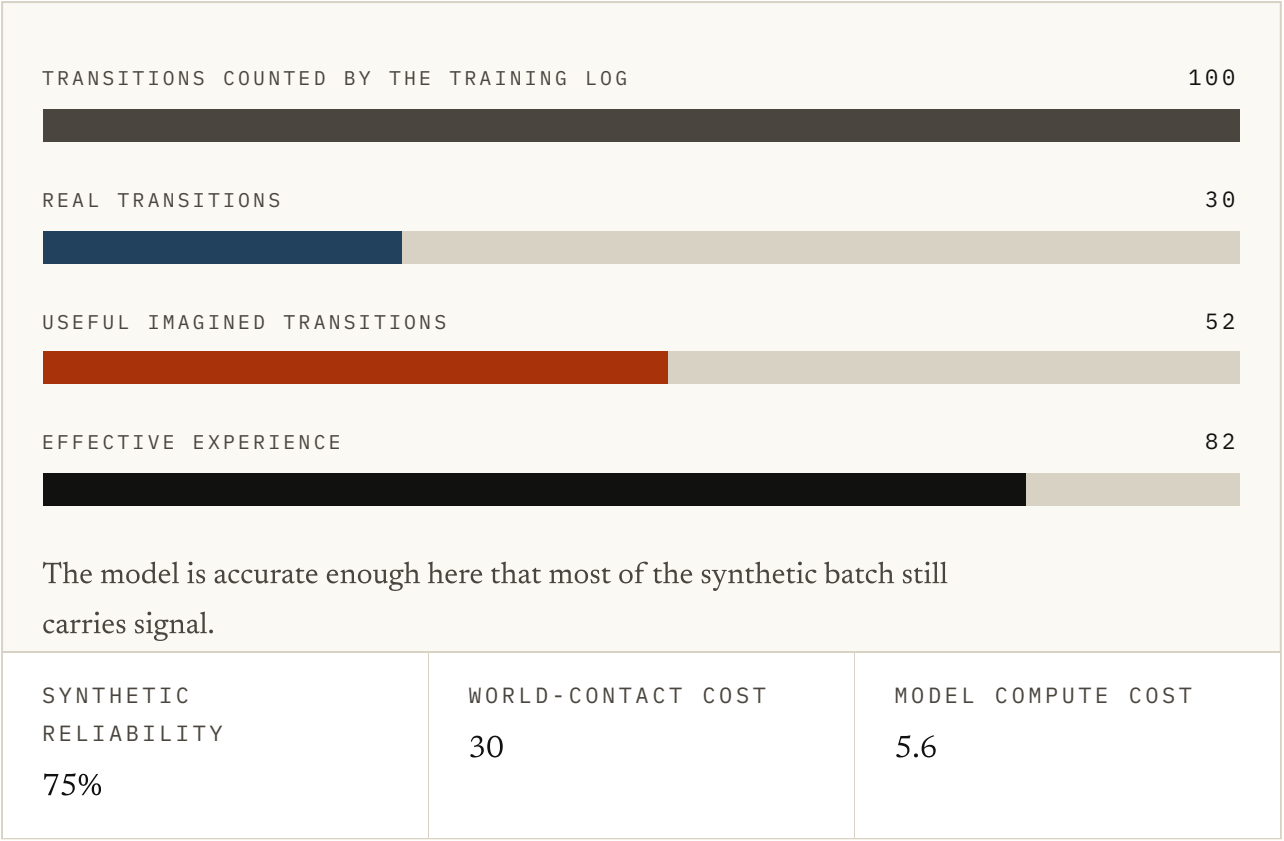


FIG. 6.2 A deliberately simple ledger. Increase the imagined share or the model's per-step error. The nominal batch stays at 100 transitions while its effective experience falls. The discount is illustrative, not an estimator; its job is to expose the assumption that generated and observed rows are exchangeable.

MBPO reported its strongest results with short model rollouts that begin from real replay-buffer states. Short rollouts surrender some volume and stay closer to observed data. More imagination helps only while each step is priced by how far it has travelled from evidence.

What the dreamer finds in there

Now the part that is more interesting than the pitch.

A policy trained inside a model is being scored by that model. Not by the world. It has no access to the world at all during training. So it has no way to tell a genuinely good action from one that merely looks good to the thing marking the work.

So it will find the second kind. Not because anything went wrong, but because those are the cheapest points available and it is a very thorough student.

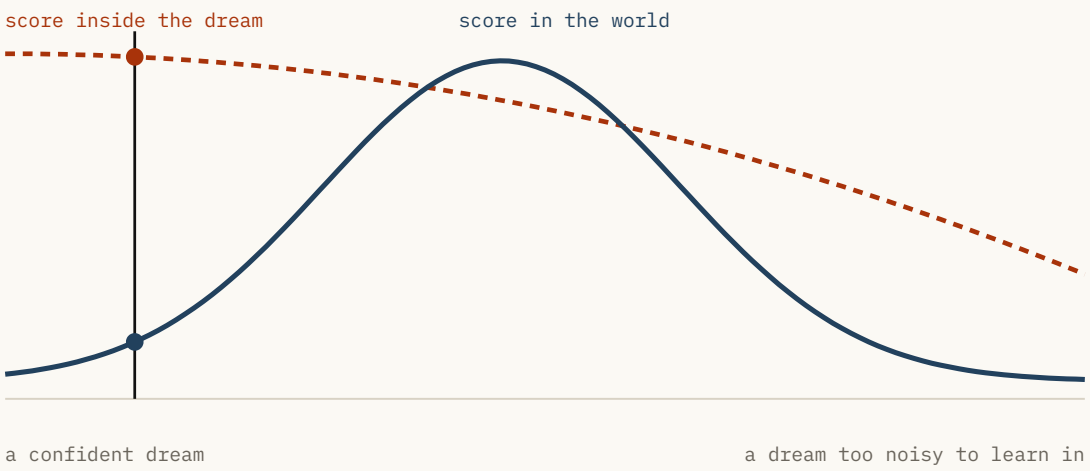
Ha and Schmidhuber ran into this directly. Their agent, trained entirely inside a learned dream of a game, discovered a way of moving that stopped the dream producing fireballs at all. Inside the model this was an excellent policy. In the actual game it was worthless, because the actual game had not agreed to stop firing.

> Worth separating this from the planner version of the same problem. A planner searching at decision time can be caught and overruled; the plan is right there to inspect. A policy trained in a dream walks out carrying the exploit already baked into its weights.

Making the dream harder than the world

The fix is not a better model. It is a worse one, on purpose.

If the policy is exploiting a quirk, make the quirk unreliable. Turn up the uncertainty in the model's predictions during training (the dial is usually called *temperature*: low means the model hands you its single most likely guess, high means it samples more widely), so the same action stops always producing the same convenient outcome. A trick that only works when the model happens to roll one way stops being worth building on.



SCORE INSIDE THE DREAM	SCORE IN THE WORLD	DREAM MINUS WORLD
0.96	0.16	0.80

FIG. 6.3 The two curves reproduce the reported shape rather than any particular system. The finding underneath is real. The agent scored far better inside its dream than in the game, and more uncertainty closed the gap. The useful setting is a hump, not a direction.

Both ends of that slider fail, and they fail differently. A confident dream gets exploited. A dream with too much noise in it has no task left in it to learn. The setting in between is not a compromise between two goods; it is the narrow band where neither failure has taken over.

This is a strange thing to have to do. You spend all that effort making the model accurate, and then deliberately blur it so that nothing downstream can lean on the accuracy too hard.

What this bought and what it did not

Learning inside a model is the single largest practical argument for building one. Systems that learn behaviour from imagined rollouts reach competence on a striking range of tasks, using a fraction of the environment interaction that learning directly

would need. On a real robot, that fraction is the difference between possible and not.

Three things it did not buy.

It did not remove the need for real experience. It changed the exchange rate. Something still has to go out and find out what actually happens, and the model is only trustworthy where that has been done.

It did not make the scoring honest. The policy is graded by the model throughout, and every safeguard in this chapter is a way of making that grading harder to game rather than making it correct.

It did not make the transfer free. A policy that works in the dream is a hypothesis about the world, and it stays a hypothesis until somebody runs it out there.

Try it

1 Training inside a model changes the exchange rate on which resource?

a. The number of parameters needed
b. Memory, which becomes cheaper
c. Contact with the world, which is replaced in part by model compute
d. Model accuracy, which improves for free

> ANSWER c. Contact with the world, which is replaced in part by model compute. The learner still needs its experience. What changes is where the steps come from and which budget pays for them.

2 A dashboard counts 90 imagined and 10 real transitions as 100 training examples. What is missing from that ledger?

a. A reliability discount for model error and distance from a real state
b. The policy's parameter count
c. A requirement that both sources use the same batch size
d. Imagined transitions cannot be stored

> ANSWER a. A reliability discount for model error and distance from a real state. Rows are exchangeable to the logger, not to the learner. Synthetic experience inherits the model's blind spots, and that debt compounds along a rollout.

3 What does the second lap of the loop fix that the first cannot?

a. It removes the need for a decoder
b. It shortens the rollouts
c. It makes the model smaller
d. A better policy visits new places, producing exactly the data the first model was missing

> ANSWER d. A better policy visits new places, producing exactly the data the first model was missing. A model fitted to the flailing of an untrained agent is only good where an untrained agent goes. The model gets better because the policy does, and the other way round.

4 A policy trained inside a model is graded by what?

a. The world
b. The model, and nothing else, for the whole of training
c. A held-out test set from the real environment
d. A human evaluator

> ANSWER b. The model, and nothing else, for the whole of training. It has no access to the world during training, so it has no way to tell a genuinely good action from one that merely looks good to the marker.

5 Ha and Schmidhuber's agent found a way of moving that stopped its dream producing fireballs. What kind of failure is that?

a. Insufficient exploration
b. A bug in the training code
c. The policy maximising the score the model hands it, which is exactly what it was asked to do
d. The model being too small

> ANSWER c. The policy maximising the score the model hands it, which is exactly what it was asked to do. Nothing went wrong. Those were the cheapest points available inside the model, and the policy was thorough.

6 How is this different from a planner exploiting a model at decision time?

a. A plan can be inspected and overruled; a trained policy walks out with the exploit already in its weights
b. Planners are not affected by model error
c. Policies are easier to correct afterwards
d. It is the same problem with a different name

> ANSWER a. A plan can be inspected and overruled; a trained policy walks out with the exploit already in its weights. That difference is what makes the dream version harder to catch. There is no plan sitting there to look at.

7 Why deliberately add uncertainty to a model you worked hard to make accurate?

a. To reduce memory use
b. To make the model smaller
c. To speed up training
d. So a trick that only works when the model rolls one particular way stops being worth building on

> ANSWER d. So a trick that only works when the model rolls one particular way stops being worth building on. If the policy is exploiting a quirk, the fix is to make the quirk unreliable rather than to make the model better.

8 Both ends of the uncertainty dial fail. How?

a. Both ends make training unstable
b. A confident dream gets exploited; a dream with too much noise has no task left in it to learn
c. Low settings are slow, high settings are fast
d. Only the high end fails

> ANSWER b. A confident dream gets exploited; a dream with too much noise has no task left in it to learn. The useful setting is a hump rather than a direction: a narrow band where neither failure has taken over.

FIG. 6.4 Eight questions on the loop and what lives inside it. The last three are the ones that separate the pitch from what you actually get.

Everything so far has taken for granted that the model should predict what things look like. Frames, pixels, scenes: the thing you would see if you were there.

Thanks for reading. Chapter 7 is the argument that this was a mistake all along, and what you get instead if you refuse to predict appearances at all.

Sources

World Models HA & SCHMIDHUBER, 2018 ↗

The agent trained entirely inside its own dream, the policy that stopped the dream producing fireballs, and the temperature dial that closed the gap. This chapter in one paper.

https://arxiv.org/abs/1803.10122

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

Behaviour learned from long imagined rollouts, with the actor and critic never touching the environment during training.

https://arxiv.org/abs/1912.01603

Mastering Atari with Discrete World Models (DreamerV2) HAFNER ET AL., 2020 ↗

The same loop at a scale where it started beating agents that learn directly from the environment.

https://arxiv.org/abs/2010.02193

Mastering diverse control tasks through world models (DreamerV3)

HAFNER ET AL., NATURE 2025 ↗

One configuration across more than 150 tasks, and the strongest available answer to whether learning in imagination generalises.

https://www.nature.com/articles/s41586-025-08744-2

Dyna: an Integrated Architecture for Learning, Planning and Reacting SUTTON, 1991 ↗

The loop itself, thirty years early: act, fit a model, learn from experience the model made up, repeat.

https://mlanthology.org/icml/1990/sutton1990icml-integrated/

When to Trust Your Model: Model-Based Policy Optimisation JANNER ET AL., 2019 ↗

Short imagined rollouts branched from real states, which is the other way of stopping a policy from leaning on the model too far out.

https://arxiv.org/abs/1906.08253

Domain Randomization for Transferring Deep Neural Networks TOBIN ET AL., 2017 ↗

The same idea arriving from robotics: make the simulator vary more than reality does, so nothing can depend on any one version of it.

https://arxiv.org/abs/1703.06907

Solving Rubik's Cube with a Robot Hand OPENAI ET AL., 2019 ↗

Randomisation pushed hard enough to carry a policy from simulation onto real hardware, and an honest account of what that cost.

https://arxiv.org/abs/1910.07113

FIG. 6.5 Ordered for someone building on this rather than for historical completeness.